VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING

Signal Processing

Laboratory Report: Lab 1 & Lab 2

Instructor: Phạm Công Thái

Student: Lê Tiến Phát – 2252600

HO CHI MINH CITY, MARCH 2026

Contents

I Lab 1 – Analog and Discrete-Time Signals **2**

1 Exercise 1.1 – Vector and Matrix Operations **2**
1.1 Task 1: Vector Addition . . . 2
1.2 Task 2: Element-wise Multiplication . . . 2
1.3 Task 3: Sine of Linearly Spaced Vector . . . 3

2 Exercise 1.2 – Analog Signal Sampling and Quantization **4**
2.1 Part (a): Analog Signal Over 5 Periods . . . 4
2.2 Parts (b) and (c): Discrete-Time Signal and Periodicity . . . 4
2.3 Part (d): Quantization with Truncation . . . 4

II Lab 2 – Discrete-Time Signals and Systems **6**

3 Exercise 1 – Investigating Scilab Functions **6**

4 Exercise 2 – Unit Step Signal **8**

5 Exercise 3 – Unit Impulse Signal **10**

6 Exercise 4 – Unit Ramp Signal **12**

7 Exercise 5 – Even and Odd Signal Decomposition **13**

8 Exercise 6 – Signal Addition **15**

9 Exercise 7 – Signal Multiplication **17**

10 Exercise 8 – Signal Manipulations **18**
10.1 8a. Folding: $y_1(n) = x(-n)$ . . . 18
10.2 8b. Advance: $y_2(n) = x(n+3)$ . . . 18
10.3 8c. Fold + Shift + Scale: $y_3(n) = 2\,x(-n-2)$ . . . 19

References **21**

Part I
Lab 1 – Analog and Discrete-Time Signals

1. Exercise 1.1 – Vector and Matrix Operations

This exercise demonstrates fundamental vector operations including element-wise arithmetic, element-wise multiplication, and applying trigonometric functions to linearly spaced vectors.

1.1. Task 1: Vector Addition

Create a vector of the form $(x_1 + 1,\ x_2 + 1,\ x_3 + 1,\ x_4 + 1)$ where $x = 1{:}4$.

Algorithm 1 Vector scalar addition

```
x ← [1, 2, 3, 4]
result ← x + 1                    ▷ Element-wise: adds 1 to every component
Display x and result
```

Explanation: The + operator applied between a vector and a scalar performs element-wise addition, adding the scalar value to every component simultaneously. With $x = [1, 2, 3, 4]$, the result is $[2, 3, 4, 5]$.

```
"Task 1: Vector x = 1:4, Result = x + 1"
 1.   2.   3.   4.
"x ="
 2.   3.   4.   5.
"x + 1 ="
```

Figure 1: Console output showing x and $x + 1$ vectors.

1.2. Task 2: Element-wise Multiplication

Create a vector of the form $(x_1y_1,\ x_2y_2,\ x_3y_3,\ x_4y_4)$ where $x = 1{:}4$ and $y = 5{:}8$.

Algorithm 2 Element-wise (Hadamard) multiplication

```
x ← [1, 2, 3, 4]
y ← [5, 6, 7, 8]
result ← x ⊙ y                    ▷ ⊙ denotes element-wise product
Display x, y, and result
```

Explanation: The element-wise product multiplies corresponding components of the two vectors, yielding $[1 \times 5,\ 2 \times 6,\ 3 \times 7,\ 4 \times 8] = [5, 12, 21, 32]$.

```
"Task 2: Element-wise product of x=1:4 and y=5:8"
 1.   2.   3.   4.
"x ="
 5.   6.   7.   8.
"y ="
 5.   12.   21.   32.
"x .* y ="
```

Figure 2: Console output showing x, y, and element-wise product.

1.3. Task 3: Sine of Linearly Spaced Vector

Create a vector $(\sin(x_1), \sin(x_2), \ldots, \sin(x_{10}))$ where x contains 10 values linearly spaced in $[0, \pi]$.

Algorithm 3 Sine of linearly spaced vector

```
x ← LINSPACE(0, π, 10)        ▷ 10 equally spaced values in [0, π]
result ← sin(x)               ▷ Element-wise sine
Display x and result
```

Explanation: The LINSPACE function generates 10 equally spaced values between 0 and π inclusive. The sin function then operates element-wise, producing values that start at $\sin(0) = 0$, peak near $\sin(\pi/2) = 1$ at the midpoint, and return to $\sin(\pi) \approx 0$ at the end.

```
"Task 3: sin(x) for 10 linearly spaced values in [0, pi]"
       column 1 to 7
 0.   0.3490659   0.6981317   1.0471976   1.3962634   1.7453293   2.0943951
       column 8 to 10
 2.443461   2.7925268   3.1415927
"x ="
       column 1 to 7
 0.   0.3420201   0.6427876   0.8660254   0.9848078   0.9848078   0.8660254
       column 8 to 10
 0.6427876   0.3420201   1.225D-16
"sin(x) ="
```

Figure 3: Console output showing the linspace vector and its sine values.

2. Exercise 1.2 – Analog Signal Sampling and Quantization

2.1. Part (a): Analog Signal Over 5 Periods

Algorithm 4 Plot analog signal over 5 periods

```
F_0 ← 100                                   ▷ Analog frequency in Hz
A ← 3,   φ ← π/3                            ▷ Amplitude and phase
T_0 ← 1/F_0                                 ▷ Period = 0.01 s
t ← LINSPACE(0, 5T_0, 1000)                 ▷ Fine time axis for smooth curve
x_a(t) ← A cos(2πF_0 t + φ)
Plot x_a(t) versus t as a continuous curve
```

Explanation: A fine time axis of 1000 points spanning 5 complete periods ensures a smooth continuous-looking waveform. The period is $T_0 = 1/F_0 = 10$ ms, so the plot covers 0 to 50 ms.

2.2. Parts (b) and (c): Discrete-Time Signal and Periodicity

Sampling $x_a(t)$ at $F_s = 300$ samples/s yields:

$$x(n) = 3\cos\left(\frac{2\pi \cdot 100}{300}\, n + \frac{\pi}{3}\right) = 3\cos\left(\frac{2\pi}{3}\, n + \frac{\pi}{3}\right)$$

The digital frequency is $f_0 = F_0/F_s = 1/3$ cycles/sample, which is rational, so $x(n)$ is **periodic** with fundamental period $N_0 = 3$ samples.

Algorithm 5 Sample analog signal and check periodicity

```
F_s ← 300                                   ▷ Sampling rate in samples/s
f_0 ← F_0/F_s                               ▷ Digital frequency = 1/3
ω_0 ← 2πf_0                                 ▷ Digital angular frequency = 2π/3
Express f_0 as irreducible fraction p/q     ▷ f_0 = 1/3 ⇒ p = 1, q = 3
if f_0 is rational then
    N_0 ← q                                 ▷ Fundamental period in samples
    Report: x(n) is periodic with period N_0
n ← [0, 1, ..., 5N_0 − 1]                   ▷ 5 periods worth of samples
x(n) ← A cos(ω_0 n + φ)
Plot x(n) as a stem plot
```

2.3. Part (d): Quantization with Truncation

Algorithm 6 Truncation quantization

```
Δ ← 1                                       ▷ Quantization step size
for each sample x(n) do
    x_q(n) ← TRUNCATE(x(n)/Δ) × Δ           ▷ Round toward zero
Plot x_q(n) as a stem plot
```

Explanation: The TRUNCATE function rounds each sample toward zero, which is distinct from floor or round. All three subplots—$x_a(t)$, $x(n)$, and $x_q(n)$—are displayed in a single figure window.

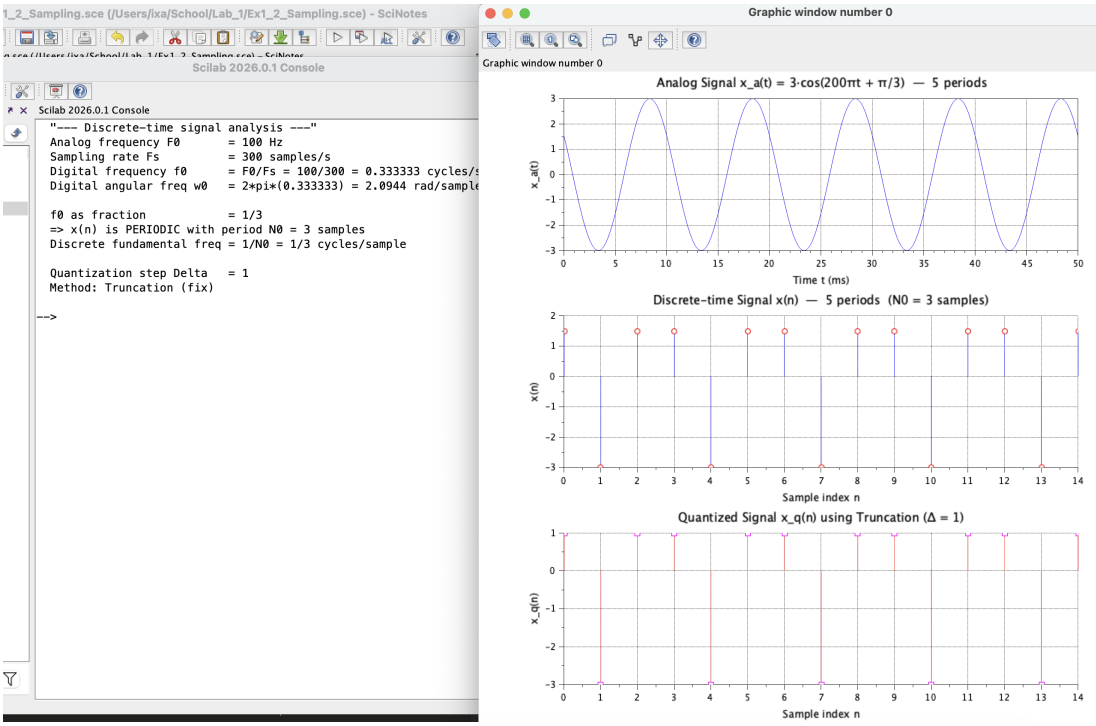

Figure 4: Single figure window with three subplots: analog $x_a(t)$ (top), discrete $x(n)$ (middle), quantized $x_q(n)$ (bottom).

Part II
Lab 2 – Discrete-Time Signals and Systems

3. Exercise 1 – Investigating Scilab Functions

The following table summarizes the functionality and usage of key Scilab functions explored in this exercise.

Function	Description
`plot2d3(x,y)`	Draws a stem (vertical bar) plot for discrete-time signals, where each data point is represented by a vertical line from the x-axis to the sample value.
`min(x)`	Returns the minimum value found in a vector or matrix.
`max(x)`	Returns the maximum value found in a vector or matrix.
`subplot(m,n,p)`	Divides the figure window into an $m \times n$ grid and activates the p-th subplot for the next plotting command.
`title(str)`	Sets the title text displayed above the current plot or subplot.
`xlabel(str)`	Sets the label for the horizontal (x) axis.
`ylabel(str)`	Sets the label for the vertical (y) axis.
`bool2s(cond)`	Converts a boolean expression into numerical values (1 for true, 0 for false), essential for constructing elementary signals like the unit step and impulse.
`deff(sig,expr)`	Defines an inline function from string arguments without requiring an external file.

Table 1: Summary of Scilab functions investigated in Exercise 1.

Algorithm 7 Demonstrate Scilab functions

$n \leftarrow [0, 1, 2, 3, 4, 5], \quad x \leftarrow [1, 3, 2, 5, 4, 6]$
$\textsc{StemPlot}(n, x)$ in subplot $(2, 2, 1)$
$v \leftarrow [3, -1, 7, 2, -5, 10]$
Print $\min(v)$ and $\max(v)$ ▷ -5 and 10
$\textsc{StemPlot}(n, 2x)$ in subplot $(2, 2, 2)$
$\textsc{StemPlot}(n, -x)$ in subplot $(2, 2, 3)$ with axis labels
$n_2 \leftarrow [-3, \ldots, 3]$
$result \leftarrow \textsc{BoolToNum}(n_2 \geq 0)$ ▷ 1 where true, 0 where false
$\textsc{StemPlot}(n_2, result)$ in subplot $(2, 2, 4)$
Define inline function: $square(x) = x^2$
Print $square(5)$ ▷ $= 25$

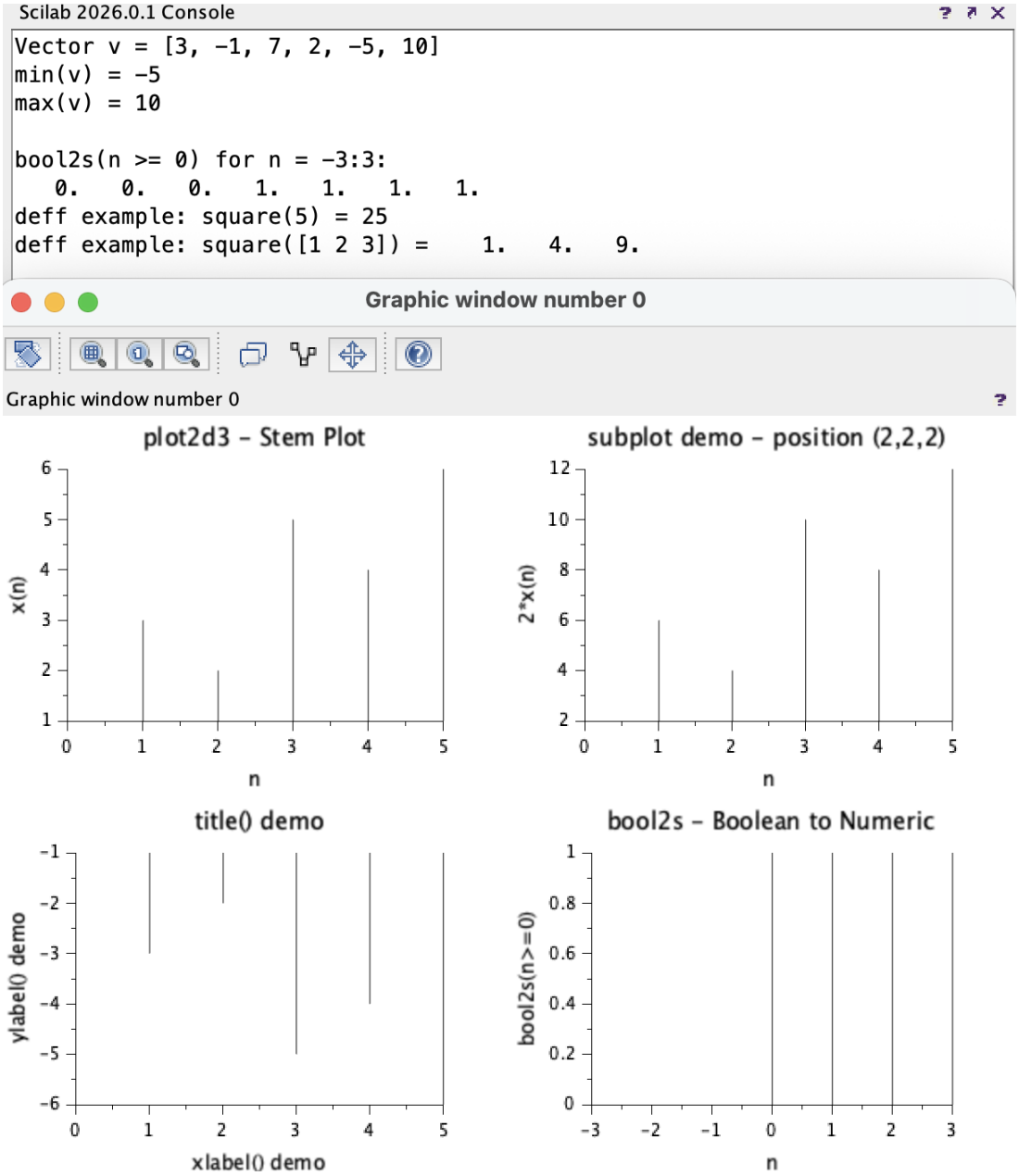

Figure 5: Console output (min, max, deff) and 2×2 subplot figure window.

4. Exercise 2 – Unit Step Signal

Algorithm 8 Generate unit step signal $u(n)$

```
n ← [−5, −4, ..., 5]
for each index n_i do
    if n_i ≥ 0 then
        u(n_i) ← 1
    else
        u(n_i) ← 0
StemPlot(n, u)
```

Explanation: This implements the unit step function defined as:

$$u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

In Scilab, `bool2s(n >= 0)` achieves this in a single vectorized operation.

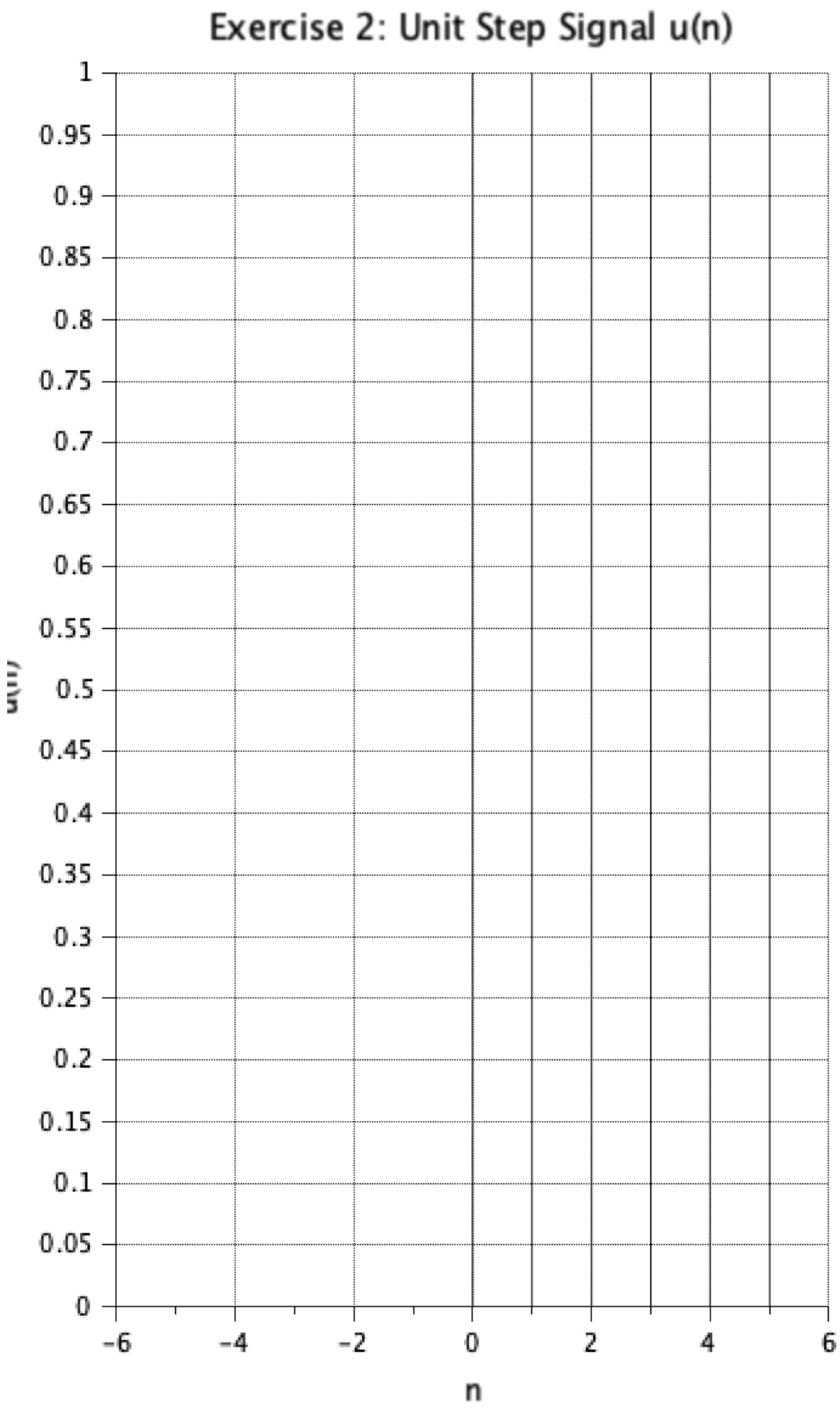

Figure 6: Stem plot of unit step signal $u(n)$ for $n = -5$ to 5.

5. Exercise 3 – Unit Impulse Signal

Algorithm 9 Generate unit impulse signal $\delta(n)$

```
n ← [−5, −4, ..., 5]
for each index n_i do
    if n_i = 0 then
        δ(n_i) ← 1
    else
        δ(n_i) ← 0
StemPlot(n, δ)
```

Explanation: This implements the Kronecker delta:

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

In Scilab, `bool2s(n == 0)` produces this in one vectorized call.

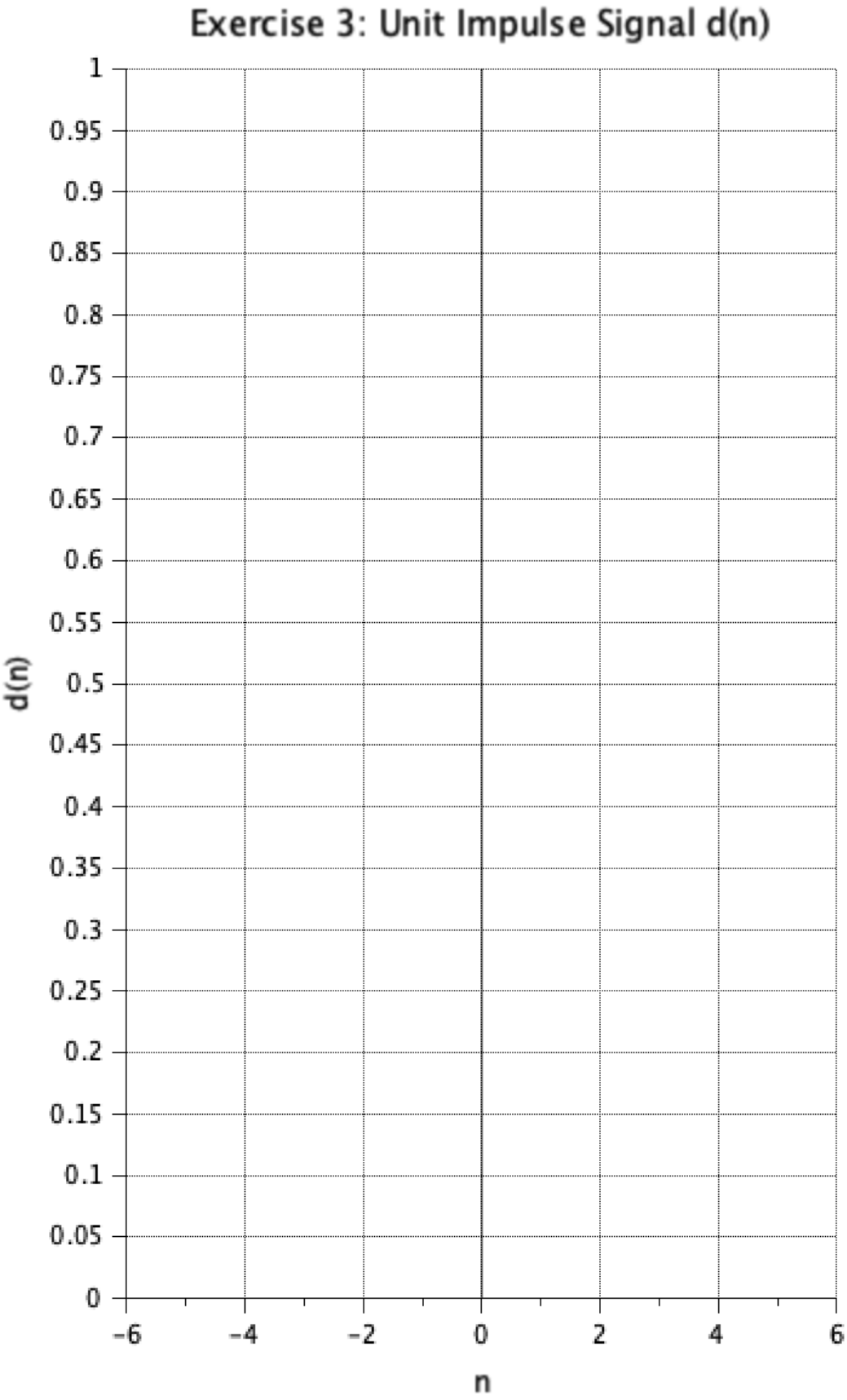

Figure 7: Stem plot of unit impulse signal $\delta(n)$ for $n = -5$ to 5.

6. Exercise 4 – Unit Ramp Signal

The unit ramp signal is defined as $u_r(n) = n \cdot u(n)$, yielding $u_r(n) = n$ for $n \geq 0$ and $u_r(n) = 0$ for $n < 0$.

Algorithm 10 Generate unit ramp signal $u_r(n)$

```
n ← [−5, −4, ..., 5]
for each index n_i do
    u_r(n_i) ← n_i × u(n_i)          ▷ Product of index and unit step
StemPlot(n, u_r)
```

Explanation: Multiplying the index n element-wise with the unit step zeroes out all negative indices while preserving the linearly increasing values for $n \geq 0$, producing $[0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 5]$.

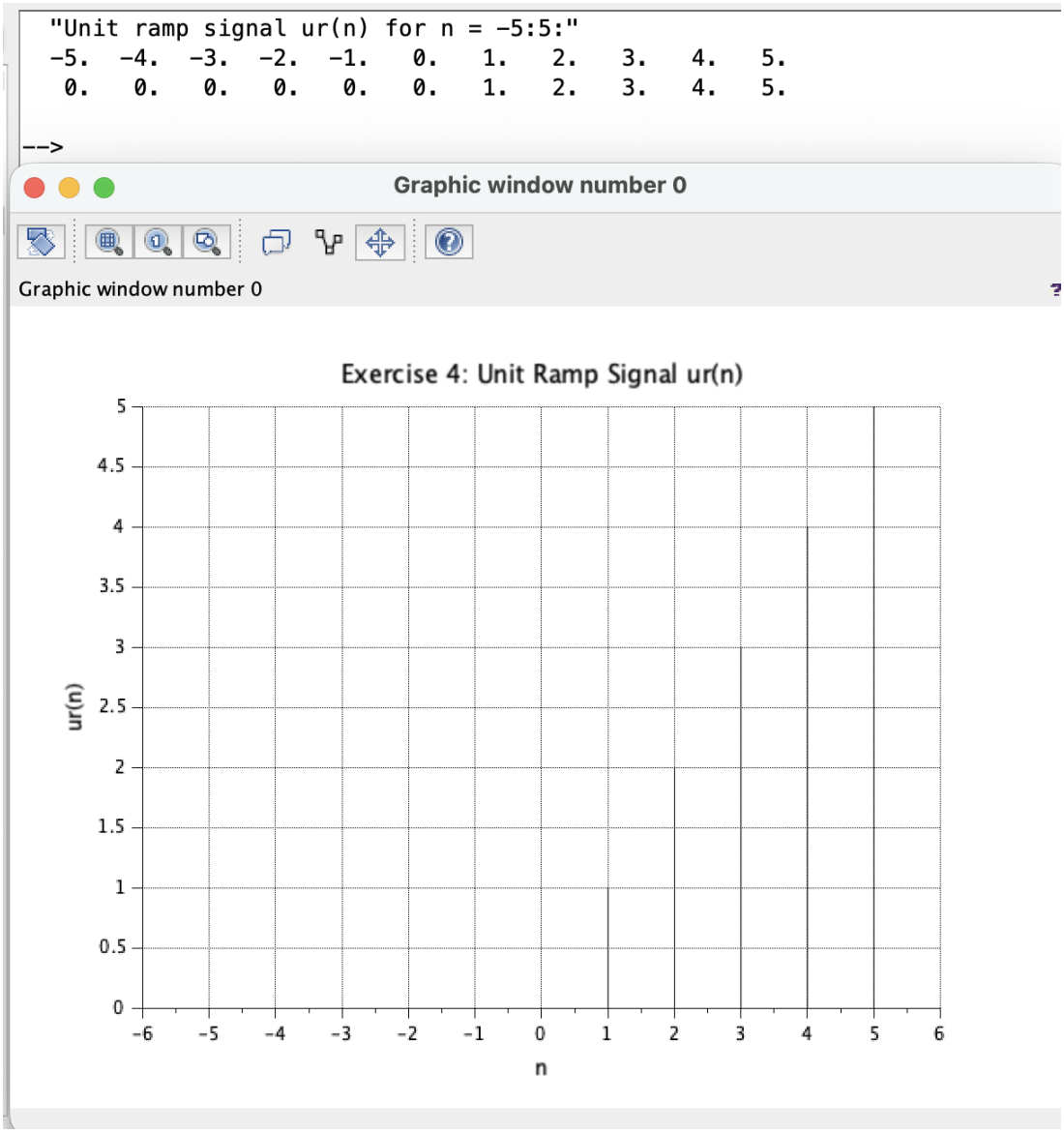

Figure 8: Stem plot of unit ramp signal for $n = -5$ to 5.

7. Exercise 5 – Even and Odd Signal Decomposition

Given $x(n) = \{1,\ \underset{\uparrow}{3},\ -2\}$ where the origin ($n = 0$) is at value 3, so $x(-1) = 1$, $x(0) = 3$, $x(1) = -2$. Any discrete signal decomposes uniquely into:

$$x_e(n) = \frac{x(n) + x(-n)}{2}, \qquad x_o(n) = \frac{x(n) - x(-n)}{2}, \qquad x(n) = x_e(n) + x_o(n)$$

Algorithm 11 Even/odd decomposition of $x(n)$

```
n ← [−1, 0, 1]
x ← [1, 3, −2]                          ▷ x(−1) = 1, x(0) = 3, x(1) = −2
x_flip ← REVERSE(x)                     ▷ x(−n) = [−2, 3, 1]
x_e ← (x + x_flip) / 2                  ▷ Even component
x_o ← (x − x_flip) / 2                  ▷ Odd component
STEMPLOT x, x_e, and x_o in three subplots
```

Results: The even component is $x_e = [-0.5,\ 3,\ -0.5]$ and the odd component is $x_o = [1.5,\ 0,\ -1.5]$. Their sum recovers $x(n)$ exactly. The even part is symmetric ($x_e(-1) = x_e(1)$) while the odd part is anti-symmetric ($x_o(-1) = -x_o(1)$).

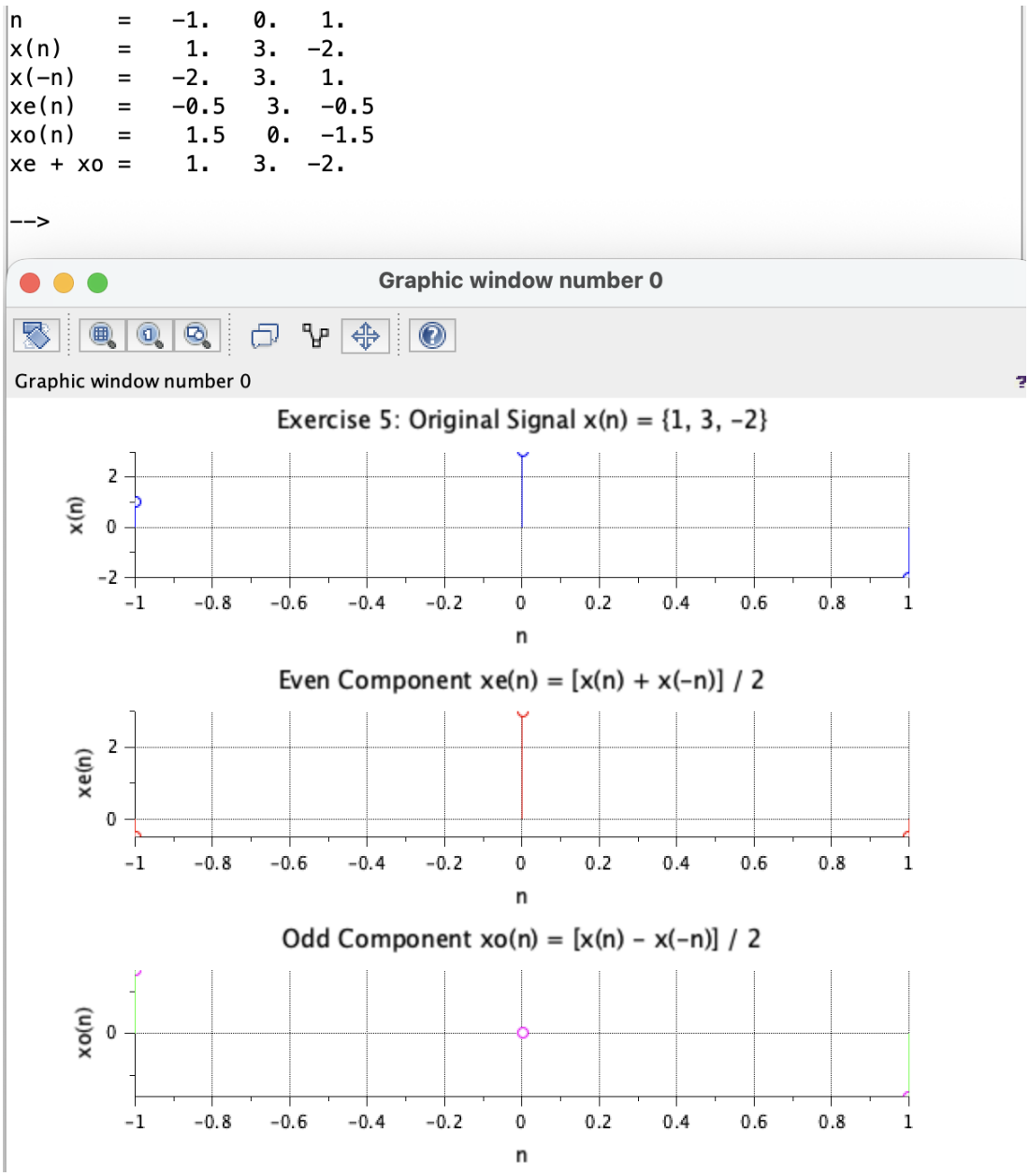

Figure 9: Three subplots: original $x(n)$, even $x_e(n)$, odd $x_o(n)$.

8. Exercise 6 – Signal Addition

Given $x_1(n) = \{\underset{\uparrow}{0},\ 1,\ 3,\ -2\}$ with $n_1 = 0{:}3$, and $x_2(n) = \{0,\ \underset{\uparrow}{1},\ 2,\ 3\}$ with $n_2 = -1{:}2$. Both signals must be aligned onto a common index range before addition.

Algorithm 12 Signal addition with index alignment

```
n_1 ← [0, 1, 2, 3],   x_1 ← [0, 1, 3, −2]
n_2 ← [−1, 0, 1, 2],   x_2 ← [0, 1, 2, 3]
n_min ← min(min(n_1), min(n_2))                ▷ = −1
n_max ← max(max(n_1), max(n_2))                ▷ = 3
n ← [n_min, ..., n_max]                        ▷ Common range [−1, 0, 1, 2, 3]
Zero-pad x_1 and x_2 to span the common range n
y ← x_1 + x_2                                  ▷ Sample-by-sample addition
StemPlot x_1, x_2, and y in three subplots
```

Result: On $n = [-1, 0, 1, 2, 3]$, the extended signals are $x_1 = [0, 0, 1, 3, -2]$ and $x_2 = [0, 1, 2, 3, 0]$, yielding $y(n) = [0, 1, 3, 6, -2]$.

```
Common index n =   -1.   0.   1.   2.   3.
x1(n) extended =    0.   0.   1.   3.  -2.
x2(n) extended =    0.   1.   2.   3.   0.
y(n) = x1+x2  =    0.   1.   3.   6.  -2.

-->
```

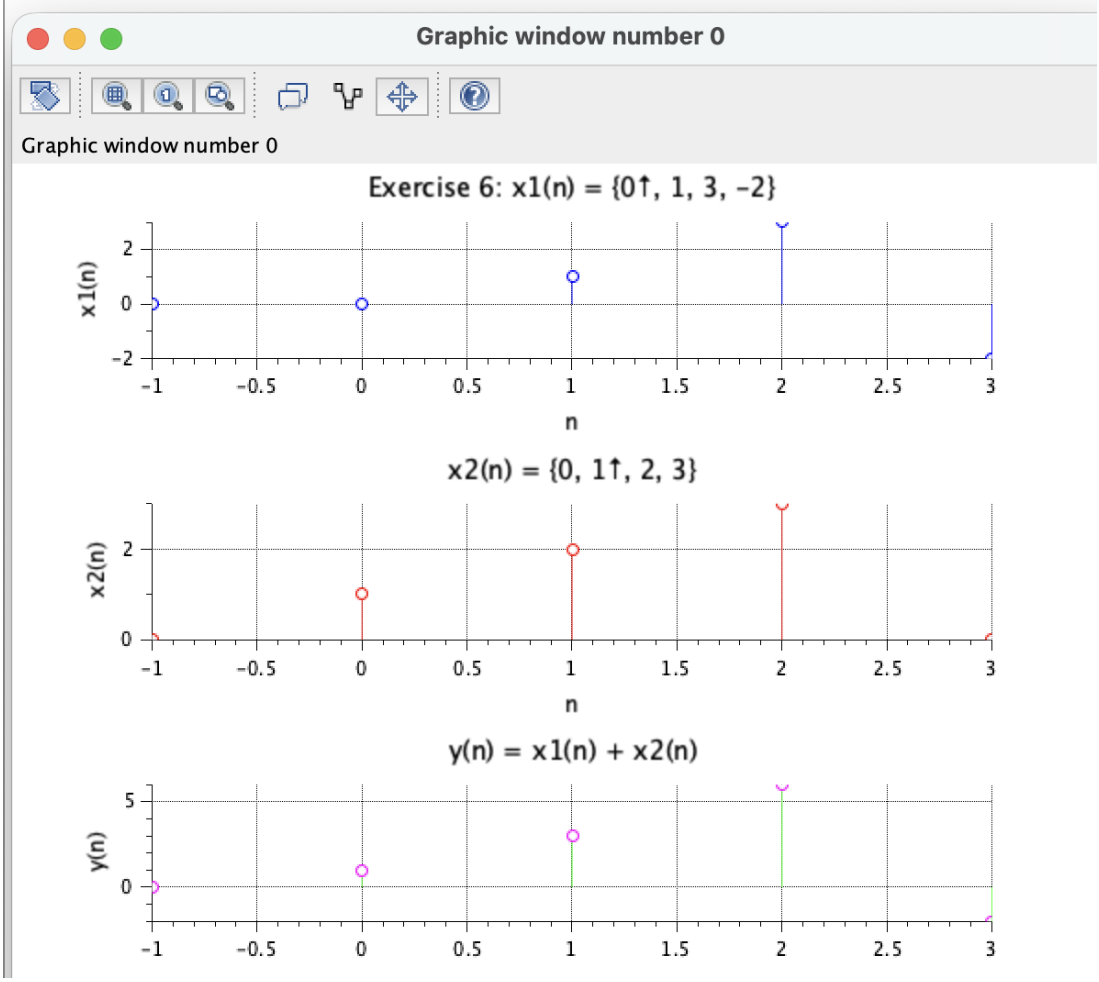

Figure 10: Three subplots: $x_1(n)$, $x_2(n)$, and sum $y(n)$.

9. Exercise 7 – Signal Multiplication

Using the same signals as Exercise 6, we compute the element-wise product $y(n) = x_1(n) \cdot x_2(n)$.

Algorithm 13 Element-wise signal multiplication

1: $n_1, x_1, n_2, x_2 \leftarrow$ same signals as Exercise 6
2: Align both onto common range $n = [-1, 0, 1, 2, 3]$ via zero-padding
3: $y(n) \leftarrow x_1(n) \odot x_2(n)$ ▷ Element-wise product
4: StemPlot x_1, x_2, and y in three subplots

Result: The element-wise product yields $y(n) = [0, 0, 2, 9, 0]$. Only $n = 1$ $(1 \times 2 = 2)$ and $n = 2$ $(3 \times 3 = 9)$ produce non-zero values, since any index where either signal is zero forces the product to zero.

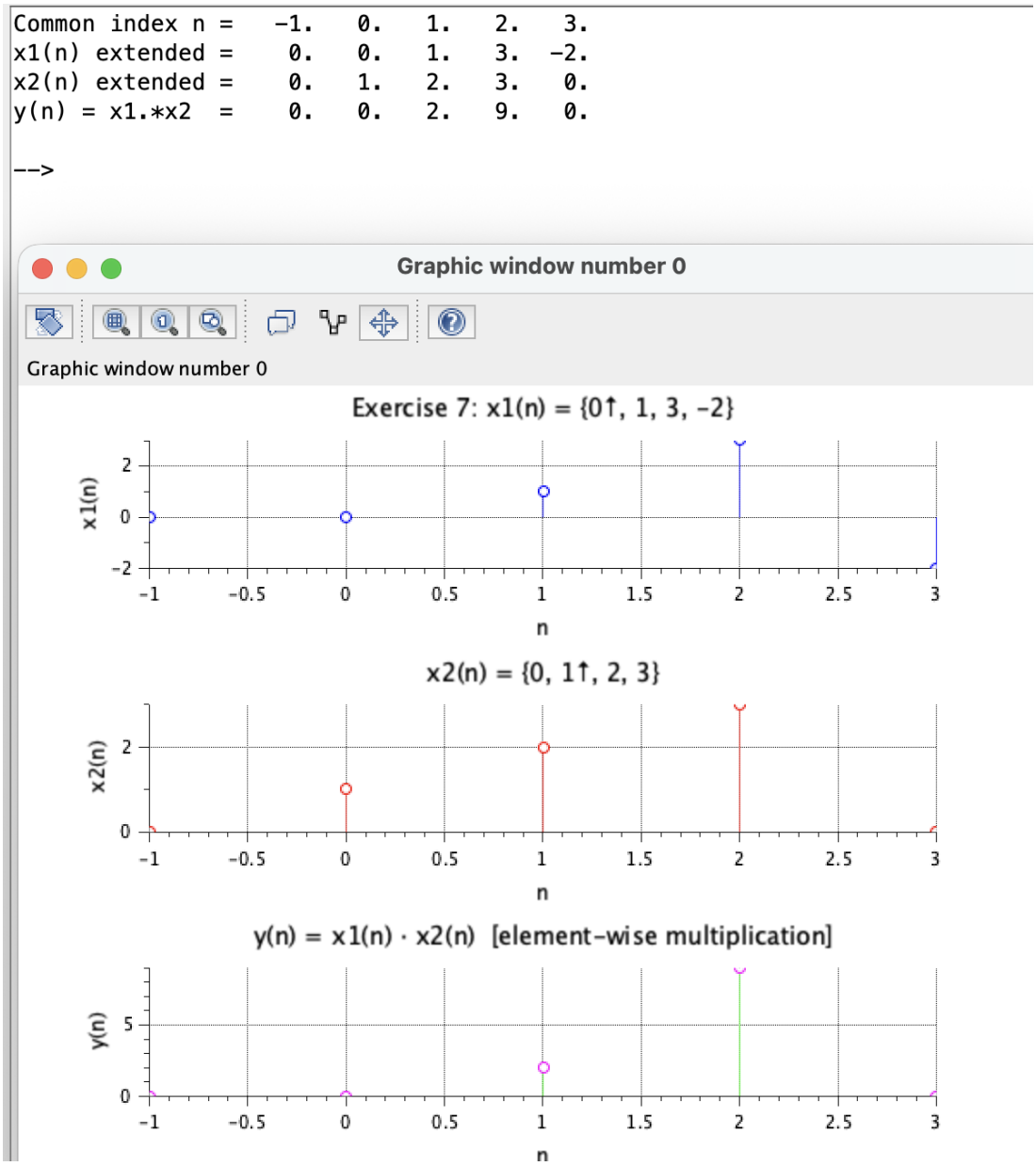

Figure 11: Element-wise multiplication result.

10. Exercise 8 – Signal Manipulations

Given $x(n) = \{1,\ -2,\ \underset{\uparrow}{3},\ 6\}$ where the origin is at the third element (value 3), so $x(-2) = 1$, $x(-1) = -2$, $x(0) = 3$, $x(1) = 6$. Three manipulations are applied as specified in the lab manual.

10.1. 8a. Folding: $y_1(n) = x(-n)$

Time reversal (folding) reflects the signal about the origin. Every sample $x(m)$ is relocated to index $-m$, which requires both reversing the value vector and negating the index vector.

Algorithm 14 Time reversal (folding)

```
n_x ← [−2, −1, 0, 1],   x ← [1, −2, 3, 6]
n_y1 ← −REVERSE(n_x)                ▷ Negate reversed indices: [−1, 0, 1, 2]
y_1 ← REVERSE(x)                    ▷ Reverse values: [6, 3, −2, 1]
STEMPLOT x and y_1 in two subplots
```

Result: The folded signal has indices $[-1, 0, 1, 2]$ with values $[6, 3, -2, 1]$, confirming that each $x(m)$ now appears at $-m$.

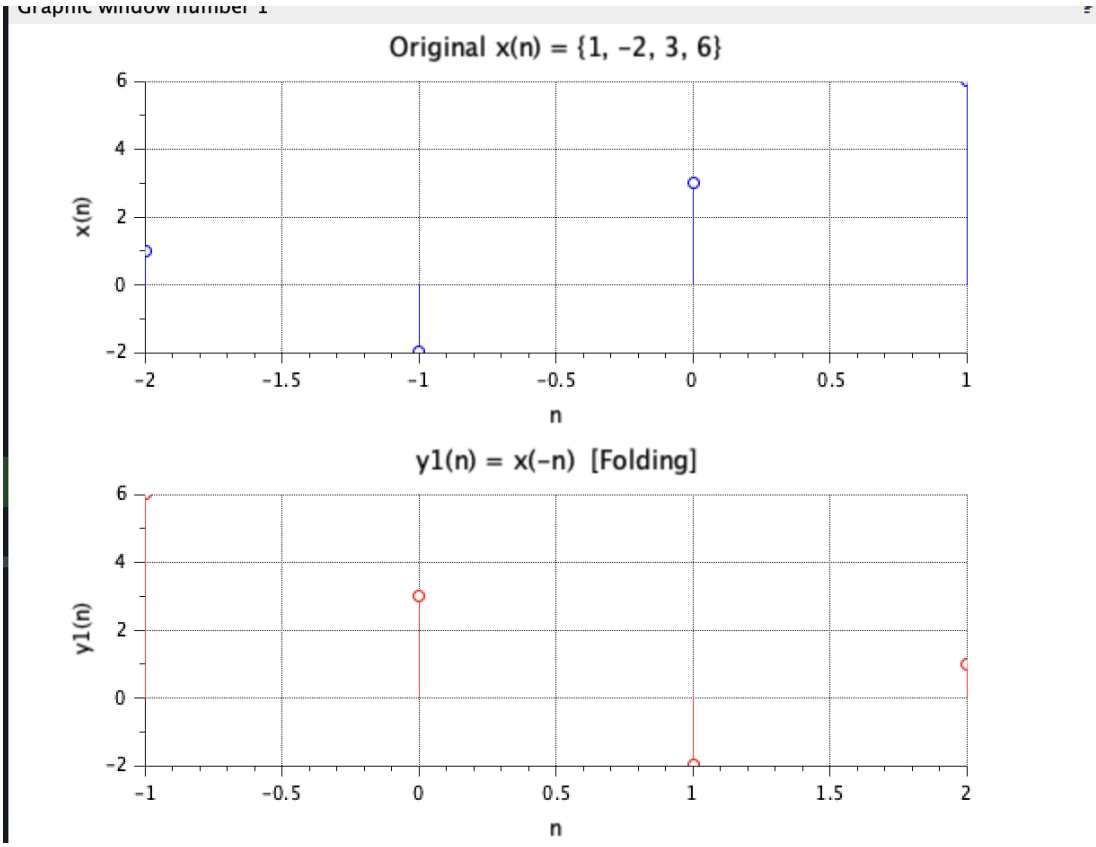

Figure 12: Original $x(n)$ (top) and folded $y_1(n) = x(-n)$ (bottom).

10.2. 8b. Advance: $y_2(n) = x(n+3)$

Advancing by k samples shifts every index to the left by k positions. If $y_2(n) = x(n+3)$, then the sample originally at index m now appears at index $m-3$, while the signal values remain

unchanged.

Algorithm 15 Advance by 3 samples

1: $n_{y_2} \leftarrow n_x - 3$ ▷ Shift indices left by 3: $[-5, -4, -3, -2]$
2: $y_2 \leftarrow x$ ▷ Values unchanged: $[1, -2, 3, 6]$
3: StemPlot x and y_2 in two subplots

Result: Indices shift from $[-2, -1, 0, 1]$ to $[-5, -4, -3, -2]$, moving the entire signal three samples earlier in time.

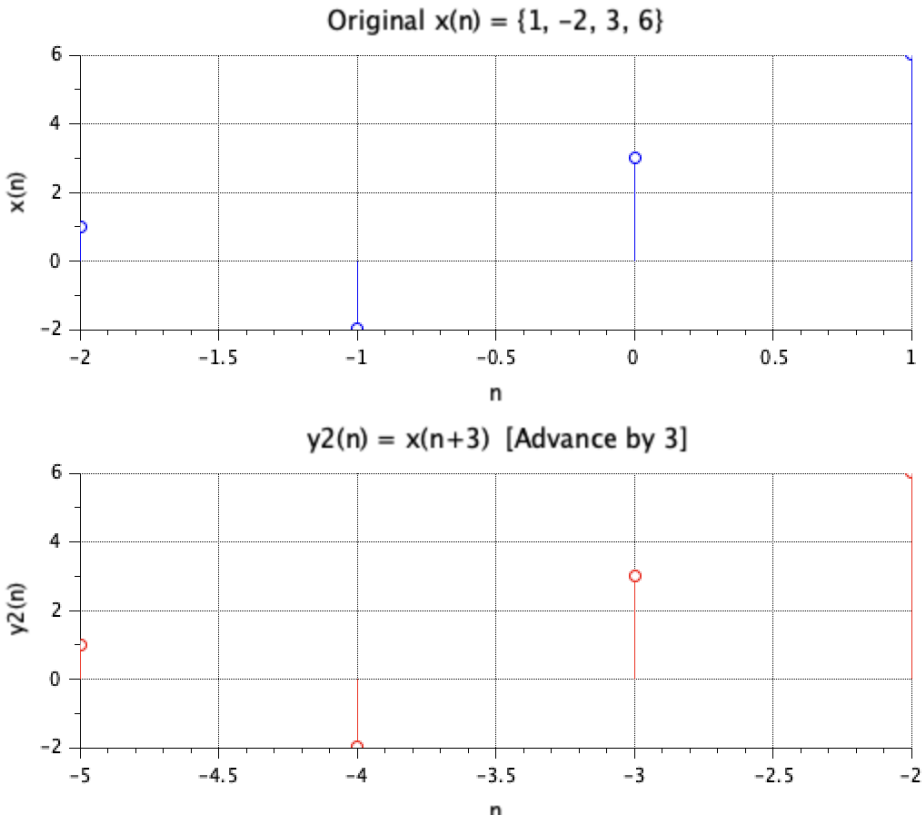

Figure 13: Original $x(n)$ (top) and advanced $y_2(n) = x(n+3)$ (bottom).

10.3. 8c. Fold + Shift + Scale: $y_3(n) = 2\,x(-n-2)$

This manipulation combines three operations applied sequentially: first fold the signal (time reversal), then shift the folded result left by 2, and finally scale all amplitudes by a factor of 2.

Algorithm 16 Combined: fold, shift, and amplitude scale

1: **Step 1 – Fold:** $n_{\text{fold}} \leftarrow -\text{Reverse}(n_x) = [-1, 0, 1, 2]$
2: $x_{\text{fold}} \leftarrow \text{Reverse}(x) = [6, 3, -2, 1]$
3: **Step 2 – Shift left by 2:** $n_{y_3} \leftarrow n_{\text{fold}} - 2 = [-3, -2, -1, 0]$
4: **Step 3 – Scale by 2:** $y_3 \leftarrow 2 \times x_{\text{fold}} = [12, 6, -4, 2]$
5: StemPlot x and y_3 in two subplots

Result: Starting from the original signal, the folding mirrors it about the origin, the shift of -2 relocates every sample two positions to the left, and the factor of 2 doubles every amplitude. The final signal at indices $[-3, -2, -1, 0]$ has values $[12, 6, -4, 2]$.

We can verify by checking individual points: $y_3(0) = 2\,x(-0-2) = 2\,x(-2) = 2 \times 1 = 2$ ✓, and $y_3(-3) = 2\,x(3-2) = 2\,x(1) = 2 \times 6 = 12$ ✓.

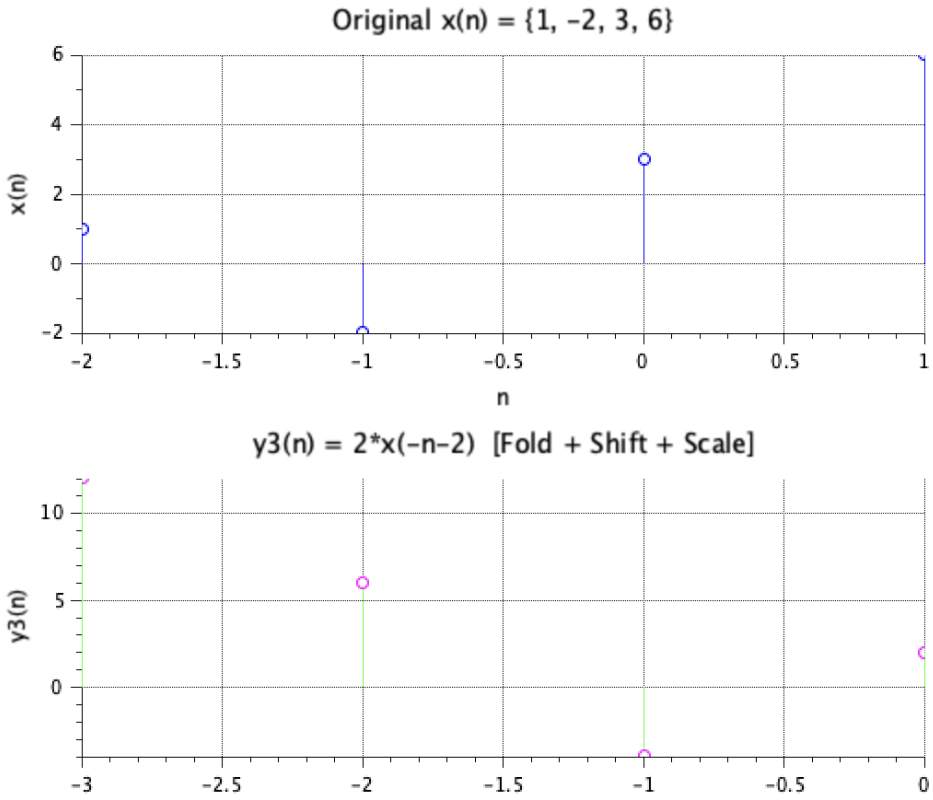

Figure 14: Original $x(n)$ (top) and $y_3(n) = 2\,x(-n-2)$ (bottom).

References

1. Scilab Online Help: `https://help.scilab.org`
2. GitHub repository with all Scilab source files: `https://github.com/Ixalis/DSP-Lab.git`

End of Report